CCF CSP 计算机软件能力认证

CCF CSP

第 39 次认证

时间：2025 年 9 月 21 日 13:30 ~ 17:30

题目名称	蒙特卡洛	水印检查	HTTP 头信息	造题计划（上）	造题计划（下）
题目类型	传统型	传统型	传统型	传统型	传统型
输入	标准输入	标准输入	标准输入	标准输入	标准输入
输出	标准输出	标准输出	标准输出	标准输出	标准输出
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒	5.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB	512 MiB
子任务数目	10	20	10	4	4
测试点是否等分	是	是	是	否	否

蒙特卡洛（pi）

【题目描述】

小 P 在学习了蒙特卡洛模拟后，设计了如下实验来估算 π 的值。

1. 准备阶段：对于给定参数 a，小 P 在平面直角坐标系上首先画出一个边长为 $2a$ 的正方形，左下角和右上角坐标分别为 $(-a,-a)$ 和 (a,a)；然后再画一个以坐标原点 $(0,0)$ 为圆心、半径为 a 的圆。
2. 随机采样：小 P 在正方形内（含边界）随机生成 n 个坐标点，并统计其中有多少个点位于圆形内（含边界）。
3. 结果分析：随机生成的坐标点落在圆形内的概率等于圆形的面积除以正方形的面积，所以理论上有：

$$\frac{m}{n} \approx \frac{\pi a^2}{4a^2}$$

这里我们用 m 表示落在圆内的坐标个数，进而可以使用如下公式来估算 π 的值：

$$\pi \approx \frac{4m}{n}$$

小 P 已经生成好了 n 个随机坐标点，但不清楚该如何统计落在圆内的数量，试编程帮助小 P 完成实验。

【输入格式】

从标准输入读入数据。

输入的第一行包含空格分隔的两个正整数 n 和 a，分别表示坐标数量和边长参数。

接下来 n 行，每行包含空格分隔的两个数，表示一个坐标。

【输出格式】

输出到标准输出。

输出一个实数，表示按上述方法估算的 π 的值。

【样例 1 输入】

```
4 5
3 4
-5 5
0 0
3 4
```

【样例 1 输出】

```
3.0
```

【样例 1 解释】

只有坐标点 $(-5, 5)$ 位于圆外。

【样例 2 输入】

```
5 5
3.01 3.99
-5 5
0 0
2.9 4.1
-3 -4
```

【样例 2 输出】

```
2.4
```

【样例 2 解释】

有三个坐标点位于圆内（含边界）。

【子任务】

50% 的测试点满足：所有输入的坐标均为整数；

全部的测试点满足：$0 < n, a \leq 1000$，所有坐标点均在正方形内（含边界）且输入的实数坐标最多包含两位小数。

【评分方式】

输出结果与标准答案相比绝对误差小于 0.0001 即可，推荐保留六位小数输出结果。

水印检查（watermark）

【题目背景】

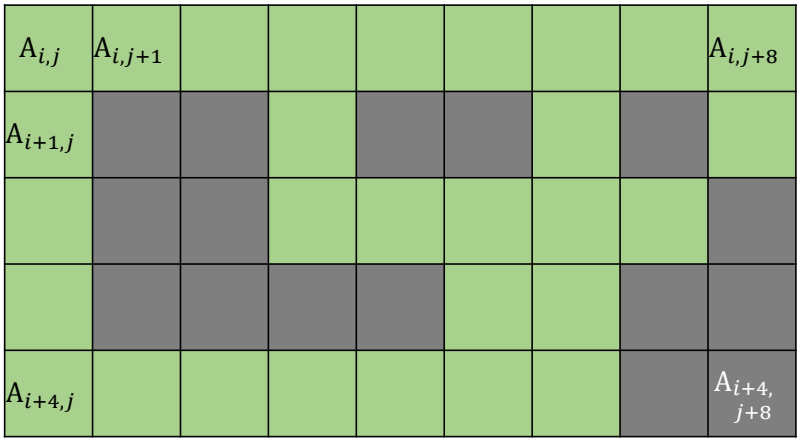
一幅长宽分别为 n 个像素和 m 个像素的灰度图像可以表示为一个 $n \times m$ 大小的矩阵 A。其中每个元素 $A_{i,j}$（$1 \le i \le n$、$1 \le j \le m$）是一个 $[0, L-1]$ 范围内的整数，表示对应位置像素的灰度值。具体来说，一个 8 比特的灰度图像中每个像素的灰度范围是 $[0, 255]$。

【题目描述】

小 P 在学习了数字图像处理后，设计了一种形如 CSP 的水印。具体来说，若想检测出水印，需选定一个阈值参数 k，然后将图像二值化：

- 灰度值大于等于 k 的像素变为白色；
- 灰度值小于 k 的像素变为黑色。

然后检查图像中是否有 5×9 的区域如下所示（为方便展示用绿色替代白色）：



即呈现出 CSP 三个字母的形状。

该水印较为简单，无需考虑旋转、翻转等复杂情况；只需检查灰度图像 A 是否包含一个大小为 5×9 的子矩阵 $A_{i,j} \cdots A_{i+4,j+8}$，其中大于等于阈值 k 和小于的像素分布与上图完全一致。

对于给定的 $n \times n$ 大小的灰度图像 A，试计算出所有能检查出水印 CSP 的阈值 k。

【输入格式】

从标准输入读入数据。

输入的第一行包含由空格分隔的两个正整数 n 和 L，表示图像的大小和像素灰度值的范围。

接下来 n 行输入矩阵 A，其中第 i 行（$1 \le i \le n$）包含用空格分隔的 n 个整数，依次为 $A_{i,1}, A_{i,2}, \cdots, A_{i,n}$。

【输出格式】

输出到标准输出。

输出若干行，每行包含一个整数，表示一个可以检查出水印 CSP 的阈值 k。$[0, L-1]$ 范围内所有可行阈值 k 按从小到大顺序输出。

【样例输入】

```
9 256
9 9 8 8 9 9 9 8 255
9 0 0 8 0 0 7 0 8
9 0 0 8 7 9 7 7 5
9 0 0 0 0 8 7 0 0
7 7 8 7 7 8 8 6 5
6 2 2 5 1 1 5 1 6
6 2 2 6 6 6 7 5 3
6 2 2 2 1 5 8 1 1
7 7 8 7 7 8 8 2 3
```

【样例输出】

```
4
5
7
```

【样例解释】

$k = 4$ 或 5 时图像如下所示，其中 * 和 - 分别表示白色和黑色，水印出现在后五行：

```
* * * * * * * * *
* - - * - - * - *
* - - * * * * * *
* - - - - * * - -
* * * * * * * * *
* - - * - - * - *
* - - * * * * * -
* - - - - * * - -
* * * * * * * - -
```

$k = 7$ 时图像如下所示，容易发现前五行（$A_{1,1} \cdots A_{5,9}$）显现出水印 CSP：

```
* * * * * * * * *
* - - * - - * - *
* - - * * * * * -
* - - - - * * - -
* * * * * * * - -
- - - - - - - - -
- - - - - - * - -
- - - - - - * - -
* * * * * * * - -
```

【子任务】

80% 的测试点满足：$n \leq 50$、$L = 256$；

全部的测试点满足：$9 \leq n \leq 200$、$L \in \{256, 65536\}$，像素值均在 $[0, L-1]$ 范围内，且保证至少存在一个阈值可以检测出水印 CSP。

HTTP 头信息（HPACK）

【题目背景】

西西艾弗岛大数据公司的服务器每天要处理海量的 HTTP 请求。为了提高传输效率，公司决定将所有服务器的 HTTP/1.1 版本升级到 HTTP/2 版本。小 C 作为一名实习生被分配到这个项目中，负责实现 HTTP 头信息的压缩算法 HPACK。HTTP（超文本传输协议）是万维网的基础协议。HTTP/1.1 版本中，头信息使用纯文本格式传输，每个头字段占一行，字段名和字段值之间用冒号加空格分隔。HTTP/2 版本中，头信息使用 HPACK（二进制格式）传输，减少了传输的数据量。

HTTP 的头信息，是一系列有序的键值对（key-value pairs），用于描述请求或响应的元数据，其中，键是字段名，值是字段值，并且字段名是可以重复的。HPACK 引入了两张表格：静态表格和动态表格，这两张表格中都存储有一系列的字段名和字段值。静态表格是预定义的，包含了一些常用的键值对；而动态表格则是根据实际的请求和响应动态更新的。在解码开始前，动态表格为空。在解码过程中，会不断向动态表格中添加新的键值对。动态表格的大小是有限的，当动态表格的条目数超过限制时，会删除最旧的条目以腾出空间。静态表格和动态表格的条目可以出现重复，即可能存在字段名相同但字段值不同的条目；也可能存在字段名和字段值都相同的条目。静态表格和动态表格的条目从 1 开始编号。假设静态表格有 S 条目，动态表格有 D 条目，则静态表格中的条目依次编号为 $1, 2, \ldots, S$，动态表格中的条目依次编号为 $S+1, S+2, \ldots, S+D$。当向动态表格中添加新的条目时，将新的条目插入到动态表格的最前面，即编号为 $S+1$ 的位置，并将原有的条目依次向后移动一个位置。当动态表格中的条目数超过限制时，则去掉编号最大的条目。

HPACK 编码后的头信息由一系列的指令组成，每条指令表示一个头信息，将这些指令按顺序解码即可得到原始的头信息。指令分为如下几种类型：

- 表格引用指令：即直接按照编号输出静态或动态表格中的某个条目的字段名和字段值；
- 字面量并索引指令：该指令包括一个编号，如果编号为 0，则另外跟随一个字符串，表示字段名；否则表示使用静态或动态表格中对应编号的条目的字段名。该指令还包括一个字符串，表示字段值。输出该字段名和字段值后，再将该键值对插入到动态表格；
- 字面量不索引指令：该指令与字面量并索引指令类似，但不会将该键值对插入到动态表格中。

在前述指令中，出现的作为字段名或字段值的字符串，可以是原样输出的字符串，也可以是经过 Huffman 编码的字符串。

Huffman 编码是一种常用的无损数据压缩算法。它通过构造一棵二叉树来为每个字符分配一个唯一的二进制编码，并使得常用字符的编码更短，从而减少整体数据的长度。在二叉树中，每个叶子节点表示一个字符，从根节点到叶子节点的路径表示该字符的编

码：在路径上，向左走表示添加一个 0，向右走表示添加一个 1。这样，常用字符会出现在树的上层，拥有较短的编码，而不常用字符则会出现在树的下层，拥有较长的编码。

【题目描述】

为了表示一个 Huffman 编码树，对于 Huffman 树从根节点开始的每一个节点，如果这个节点是叶子节点，那么输出 1，并且输出该节点对应的字符；否则输出 0，然后递归地对该节点的左子节点和右子节点进行同样的操作。例如：

```
      *
     / \
    *   a
   / \
  b   *
     / \
    c   d
```

可以表示为 001b01c1d1a。根据该表示，即可还原出 Huffman 树。上面的树中，字符 a 的编码为 1；字符 b 的编码为 00；字符 c 的编码为 010；字符 d 的编码为 011。

对于前述三种指令，其格式约定如下：

- 表格引用指令：1 i，其中 i 是一个正整数，表示引用编号为 i 的条目；
- 字面量并索引指令：2 0 k v 或 2 i v，其中 i 是一个正整数，表示使用编号为 i 的条目的字段名；k 和 v 是字符串或 Huffman 编码的字符串，分别表示字段名和字段值；
- 字面量不索引指令：3 0 k v 或 3 i v，其中 i 是一个正整数，表示使用编号为 i 的条目的字段名；k 和 v 是字符串或 Huffman 编码的字符串，分别表示字段名和字段值。

其中，字符串是由小写字母、大写字母、数字（a-zA-Z0-9）组成的，其格式约定如下：

- 普通字符串：s，当 s 的开头不是 H 时，表示原样输出字符串 s；当 s 的开头是 HH 时，表示输出字符串 s 去掉开头的第一个 H 后的部分；
- Huffman 编码的字符串：Hb，其中 b 是一个由偶数个 0-9a-f 组成的字符串，表示原始字符串经过 Huffman 编码后得到的二进制数据序列（以十六进制显示）。其中，原始字符串靠前的字符对应的编码，存储于每一字节较高的位；每个字符的编码的高位存储于每一字节较高的位；每个字符的编码按位连续存储；最后一字节的低位补 0，并增加一字节表示补 0 的个数 p（$0 \le p \le 7$）。例如，按前述 Huffman 树对字符串 abcd 进行编码，得到的二进制数据序列为 100010011，补 0 后为 10001001 10000000，最后一字节补了 7 个 0，因此表示为 H898007。

【输入格式】

从标准输入读入数据。

输入的第一行包含两个整数 S 和 D，分别表示静态表格和动态表格的条目数。接下来 S 行，每行包含空格分隔的两个字符串，依次表示静态表格中每个条目的字段名和字段值。

接下来的一行包含一个字符串，表示 Huffman 树的表示。接下来的一行包含一个整数 N，表示指令的条数。接下来 N 行，每行表示一条指令，格式如前述所述。

【输出格式】

输出到标准输出。

输出 N 行，每行表示解码后的头信息，格式为 `key: value`，其中 `key` 和 `value` 分别表示字段名和字段值。对于输入中的“普通字符串”和“使用霍夫曼编码的字符串”，需输出解码后得到的原始字符串。

【样例 1 输入】

```
3 3
Foo Bar
Foo Barz
bar barzz1
001b01c1d1a
11
1 1
3 3 ABCD
3 0 H898007 HHIJK
2 0 H898007 EFGH
1 4
2 0 DDDD EEEE
2 5 GGGG
1 4
1 6
2 0 HHHH IIII
1 4
```

【样例 1 输出】

```
Foo: Bar
bar: ABCD
abcd: HIJK
abcd: EFGH
abcd: EFGH
DDDD: EEEE
abcd: GGGG
abcd: GGGG
abcd: EFGH
HHH: IIII
HHH: IIII
```

【样例 1 解释】

输入首先给出了静态表的内容，和动态表的大小限制 $D = 3$。接着给出了 Huffman 树的表示 `001b01c1d1a`，该表示即题目描述中的示例。接下来，输入给出了 11 个指令。其中，第一个指令表示引用静态表中编号为 1 的条目，因此直接输出 `Foo: Bar`。第二个指令表示使用静态表中第 3 项的字段名作为字段名，字段值为给定的 `ABCD`，因此输出 `bar: ABCD`。第三个指令中，字段名使用了 Huffman 编码。其中，最后的 `07` 表示最后一个字节补了 7 个 0，因此十六进制串 `8980` 对应的二进制数据序列为 `100010011`，解码后得到字符串 `abcd`。第三个指令中，字段值前有两个 `H`，需要去掉一个，因此得到输出为 `abcd: HIJK`。第四个指令与第三个指令类似，但是要将得到的键值对插入到动态表中。第五个指令引用了编号为 4 的项目，即动态表的第一项，因此输出 `abcd: EFGH`。第六个指令继续向动态表中插入新的键值对，此时动态表中已有两项。第七个指令中，字段名使用了编号为 5 的条目，即动态表的第二项。需要注意的是，每次向动态表中插入新的条目时，都将新的条目插入到动态表的最前面，并将原有的条目依次向后移动一个位置。因此，此时的动态表的第二项，即为第四个指令中插入的条目，所以输出为 `abcd: GGGG`，然后再将该条目插入到动态表中。第八个指令引用了编号为 4 的条目，即动态表的第一项，因此输出 `abcd: GGGG`。第九个指令引用了编号为 6 的条目，即动态表的第三项，因此输出 `abcd: EFGH`。第十个指令向动态表中继续插入一个条目，插入前动态表已满，因此需要在插入新条目前，先丢弃最后一个条目，再将新条目插入到动态表的最前面。第十一个指令引用了编号为 4 的条目，即动态表的第一项，因此输出 `HHH: IIII`。

【子任务】

对于 20% 的数据，仅包含表格引用指令；

对于 40% 的数据，仅包含表格引用指令和字面量不索引指令，且不包含 Huffman 编码的字符串；

对于 60% 的数据，含有所有类型的指令，且字面量并索引指令的数目不超过 D；

对于 100% 的数据，有：$1 \le S \le 64$；$1 \le D \le 120$；Huffman 树中字符的 `01` 编码长度不超过 8 位；$1 \le N \le 1000$；表示字段名和字段值的字符串在解码前后的长度均不超过 150。

【提示】

本题提供了一些大语言模型生成的代码模块辅助解题（见下发文件），你可以参考、使用这些代码，也可以忽略它们自己手动实现全部功能，详情如下。

小 C 打开了某个大语言模型，并向它提问，以下是提问和回答：

> 提问：为了表示一个 Huffman 编码树，对于 Huffman 树从根节点开始的每一个节点，如果这个节点是叶子节点，那么输出 1，并且输出该节点对应的字符；否则输出 0，然后递归地对该节点的左子节点和右子节点进行同样的操作。请根据上述表示的办法，将 Huffman 编码树恢复为树状结构。请分别给出 C++、Python、Java 语言的实现。
>
> 回答：我将为您提供 C++、Python 和 Java 三种语言的 Huffman 树恢复实现。

C++ 实现

```cpp
#include <iostream>
#include <string>
#include <memory>
using namespace std;

struct Node {
    char data;
    shared_ptr<Node> left;
    shared_ptr<Node> right;

    Node(char d) : data(d), left(nullptr), right(nullptr) {}
    Node() : data('\0'), left(nullptr), right(nullptr) {}
};

shared_ptr<Node> rebuildHuffmanTree(const string& s, int& index) {
```

```
    if (index >= s.length()) return nullptr;

    if (s[index] == '1') {
        index++; // 跳过'1'
        char ch = s[index++]; // 读取字符
        return make_shared<Node>(ch);
    } else if (s[index] == '0') {
        index++; // 跳过'0'
        auto node = make_shared<Node>();
        node->left = rebuildHuffmanTree(s, index);
        node->right = rebuildHuffmanTree(s, index);
        return node;
    }
    return nullptr;
}

// 辅助函数：打印树结构（前序遍历）
void printTree(const shared_ptr<Node>& root, string
   prefix = "") {
    if (!root) return;

    if (root->data != '\0') {
        cout << prefix << "Leaf: " << root->data << endl;
    } else {
        cout << prefix << "Internal Node" << endl;
    }
    printTree(root->left, prefix + "  ");
    printTree(root->right, prefix + "  ");
}

int main() {
    string encodedTree = "001a1b1c"; //
   示例：0表示内部节点，1表示叶子节点
    int index = 0;
    auto root = rebuildHuffmanTree(encodedTree, index);

    cout << "Rebuilt Huffman Tree:" << endl;
```

```
    printTree(root);

    return 0;
}
```

Python 实现

```python
class Node:
    def __init__(self, data=None):
        self.data = data
        self.left = None
        self.right = None

def rebuild_huffman_tree(s, index):
    if index[0] >= len(s):
        return None

    if s[index[0]] == '1':
        index[0] += 1  # 跳过'1'
        ch = s[index[0]]
        index[0] += 1  # 读取字符
        return Node(ch)
    elif s[index[0]] == '0':
        index[0] += 1  # 跳过'0'
        node = Node()
        node.left = rebuild_huffman_tree(s, index)
        node.right = rebuild_huffman_tree(s, index)
        return node
    return None

# 辅助函数：打印树结构
def print_tree(root, prefix=""):
    if root is None:
        return

    if root.data is not None:
        print(f"{prefix}Leaf: {root.data}")
    else:
```

```
        print(f"{prefix}Internal Node")
    print_tree(root.left, prefix + "  ")
    print_tree(root.right, prefix + "  ")

# 使用示例
if __name__ == "__main__":
    encoded_tree = "001a1b1c"  # 示例编码
    index = [0]  # 使用列表来传递可变索引
    root = rebuild_huffman_tree(encoded_tree, index)

    print("Rebuilt Huffman Tree:")
    print_tree(root)
```

Java 实现

```
class Node {
    Character data;
    Node left;
    Node right;

    Node(Character data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }

    Node() {
        this.data = null;
        this.left = null;
        this.right = null;
    }
}

public class HuffmanTreeRebuilder {

    public static Node rebuildHuffmanTree(String s, int[]
   index) {
        if (index[0] >= s.length()) {
```

```
            return null;
        }

        if (s.charAt(index[0]) == '1') {
            index[0]++; // 跳过'1'
            char ch = s.charAt(index[0]);
            index[0]++; // 读取字符
            return new Node(ch);
        } else if (s.charAt(index[0]) == '0') {
            index[0]++; // 跳过'0'
            Node node = new Node();
            node.left = rebuildHuffmanTree(s, index);
            node.right = rebuildHuffmanTree(s, index);
            return node;
        }
        return null;
    }

    // 辅助函数：打印树结构
    public static void printTree(Node root, String
prefix) {
        if (root == null) {
            return;
        }

        if (root.data != null) {
            System.out.println(prefix + "Leaf: " +
root.data);
        } else {
            System.out.println(prefix + "Internal Node");
        }
        printTree(root.left, prefix + "  ");
        printTree(root.right, prefix + "  ");
    }

    public static void main(String[] args) {
        String encodedTree = "001a1b1c"; // 示例编码
```

```
58          int[] index = {0}; // 使用数组来传递可变索引
59          Node root = rebuildHuffmanTree(encodedTree,
    index);
60
61          System.out.println("Rebuilt Huffman Tree:");
62          printTree(root, "");
63      }
64  }
```

造题计划（上）（SPJ）

【题目背景】

西西艾弗大学的第三十九届算法考试就要开始了，小 C 正在紧张地造题……

【题目描述】

小 C 想到了这样一个题目：给定一棵 n 个结点的树，要求选手给每个结点分配一个权值，使得树上所有点的权值构成一个 0 到 $n-1$ 的排列，并满足 m 条限制，每条限制形如 (x, y, v)，表示树上从 x 到 y 路径上所有点的权值构成集合的 mex 值等于 v。

小 C 已经想到了绝妙的正解做法，现在他需要你帮他写 SPJ 的核心部分。具体而言，你会接收到一棵 n 个结点的树，其中编号为 i 的点有权值 a_i，且 $a_1, a_2, \ldots, a_n$ 构成一个 0 到 $n-1$ 的排列。接下来你要依次检验 m 条限制，每条限制会给定树上的两个结点 x 和 y，你需要求出从 x 到 y 的简单路径上所有点的权值构成集合的 mex 值。

对于一个集合 S，$\mathrm{mex}(S)$ 表示**最小的没有在 S 中出现过的自然数**。例如：

- $\mathrm{mex}(\{0, 1, 2, 5\}) = 3$
- $\mathrm{mex}(\{1, 2\}) = 0$

【输入格式】

从标准输入读入数据。

输入的第一行包含两个正整数 n, m，分别表示用于结点数和限制条数。

第二行包含 n 个正整数 $a_1, a_2, \ldots, a_n$，表示每个结点的权值。

接下来 $n-1$ 行每行包含两个整数，表示树上的一条边。

接下来 m 行每行包含两个整数 x_i, y_i，表示一条限制。你需要求出从 x_i 到 y_i 的简单路径上所有点的权值构成集合的 mex 值。

【输出格式】

输出到标准输出。

输出 m 行每行一个正整数，第 i 行的表示第 i 条限制所需要求的 mex 值，

【样例 1 输入】

```
7 3
1 0 2 6 3 4 5
1 2
2 5
2 6
```

```
1 3
3 7
1 4
5 3
6 3
7 4
```

【样例 1 输出】

```
4
3
0
```

【子任务】

本题采用捆绑测试，你只有通过一个子任务中的所有测试点才能得到该子任务的分数。

- 子任务一（20 分）：$n, m \leq 1000$；
- 子任务二（20 分）：保证给定的树是一条链；
- 子任务三（30 分）：$n, m \leq 5 \times 10^4$；
- 子任务四（30 分）：无特殊限制。

对于所有数据，保证 $1 \leq n, m \leq 2 \times 10^5$。$a_1, a_2, \ldots, a_n$ 构成一个 0 到 $n-1$ 的排列。

造题计划（下）（Author）

【题目背景】

西西艾弗大学的第三十九届算法考试就要开始了，小 C 和小 F 二人正在紧张地筹备题目……

【题目描述】

小 C 和小 F 需要在 n 天时间内造出若干道题目用于考试。二人的分工很明确：小 C 负责造题，小 F 负责验题。显然，一个题需要先被小 C 造完，才能被小 F 验，验完之后才能上线考试。

西西艾弗大学有合理的工作制度，每一天，小 C 会在上午造题，在小 C 造完题下班之后，小 F 会在下午验题。因为不需要打卡，他们可以灵活地选择这一天是否工作。

小 C 和小 F 都很注重工作和生活的平衡，所以小 C 一天最多出一个题，小 F 一天最多验一个题。因为人的工作状态有波动，在第 i 天，小 C 造一个题需要的精力值为 a_i，小 F 验一个题需要的精力值为 b_i。

小 C 和小 F 还要一起完成某门课的大作业，所以他们不能把全部的精力都花在造题上。根据二人精准的测算，他们用于出题的总精力值不可以大于 m。

现在小 C 和小 F 想知道，在这 n 天里他们最多能造出多少题目。

【输入格式】

从标准输入读入数据。

输入的第一行包含两个正整数 n, m，分别表示用于筹备比赛的天数和可以花费的最大总精力值。

第二行包含 n 个正整数 $a_1, a_2, \ldots, a_n$，表示每天小 C 造题需要花费的精力。

第三行包含 n 个正整数 $b_1, b_2, \ldots, b_n$，表示每天小 F 验题需要花费的精力。

【输出格式】

输出到标准输出。

输出一行一个正整数，表示小 C 和小 F 最多能造出的题目数量。

【样例 1 输入】

```
5 12
3 5 7 2 9
4 2 8 3 7
```

【样例 1 输出】

```
2
```

【样例 1 解释】

小 C 在第一天和第四天造题。
小 F 在第二天和第四天验题。

【子任务】

本题采用捆绑测试，你只有通过一个子任务中的所有测试点才能得到该子任务的分数。

- 子任务一（20 分）：$n \leq 100$；
- 子任务二（30 分）：$n \leq 3000$；
- 子任务三（20 分）：$n \leq 10^5$；
- 子任务四（30 分）：无特殊限制。

对于所有数据，保证 $1 \leq n \leq 5 \times 10^5, 1 \leq a_i, b_i \leq 10^9, 1 \leq m \leq 10^{18}$。